Predicting `cp.async` Pipeline Effectiveness from Memory-Subsystem Parameters: A Cross-GPU Study

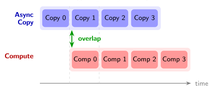
Xiangchen Song Chenxin Geng Yuyang Feng Tian Xia

EECS 570 – Parallel Computer Architecture — University of Michigan — Winter 2026

1. Problem, Motivation & Hypotheses

NVIDIA `cp.async` (Ampere+) enables *asynchronous* global→shared memory copies. CUTLASS, FlashAttention-3, and Triton use multi-stage `cp.async` pipelines; depth S is a key tuning knob.

Multi-stage `cp.async` Pipeline



The Problem: Practitioners tune S on *one* GPU and assume it transfers. Effectiveness depends on **L2 capacity**, **BW/latency**, **SMEM** — vary dramatically across GPUs.

H1 – L2-Capacity Driven — **SUPPORTED**
Speedup inflects when $W_{\text{conc}} > C_{L2}$. *L40S (112 MB): minimal benefit; A40 (8 MB): largest.*

H2 – Ridge-Point Driven — **PARTIAL**
Ridge alone insufficient — must combine with L2 + latency.

H3 – Latency–Occupancy Tension — **SUPPORTED**
S^* balances latency hiding vs. occupancy loss. *A40: non-monotonic $S^*(N)$.*

2. Parameter-Free Predictive Model

Predict `cp.async` speedup from architecture constants — **zero fitted parameters**.

Inputs per GPU (pointer-chase microbenchmark): effective L2 capacity C_{L2}^{eff}, L2 latency L_{L2}, DRAM latency L_{DRAM}.

Step 1: Effective memory latency (from **H1**):

$$W_{\text{conc}} = w_{\text{blk}} \times S \times \text{blks}_{\text{active}} \quad \text{(concurrent working set)}$$
$$h = \min\left(1,\ C_{L2}^{\text{eff}}/W_{\text{conc}}\right) \quad \text{(L2 hit fraction)}$$
$$L_{\text{eff}} = h \cdot L_{L2} + (1-h) \cdot L_{\text{DRAM}} \quad \text{(effective latency)}$$

Step 2: Compute time with floor (empirical discovery):

$$T_{\text{comp}} = \max\left(T_{\text{FLOP}},\ \mathbf{280}\right) \quad \text{(280-cycle per-stage overhead floor)}$$

Step 3: Predicted speedup (from **H3**):

$$\widehat{S}_{\text{pred}} = \underbrace{\frac{\max(T_{\text{comp}},\ L_{\text{eff}})}{\max(T_{\text{comp}},\ L_{\text{eff}}/S)}}_{\text{overlap benefit}} \times \underbrace{\left(\frac{\text{Occ}(S)}{\text{Occ}(1)}\right)^{0.15}}_{\text{occupancy penalty}}$$

- **280-cycle floor**: without it stencil MAPE = 146%; with it: **3.7%**
- α=0.15: occupancy loss highly sub-linear (3→1 blk/SM ≈ 16% loss)
- Each term derives from a hypothesis — **no curve fitting**

3. Experimental Setup & Validation

Kernels & Methodology

Kernel	Var.	Description
GEMM	V1, V3	$C = AB$, compute-bound. $N \in \{256-8192\}$, tile $\in \{16-128\}$
Stencil	V1, V3	1D, memory-bound (AI≈5). $L \in \{2^{16}-2^{26}\}$

V3: `cp.async` pipelined, $S \in \{2,3,4\}$. Median of 10 runs. cuBLAS/CPU validated. Same source on all GPUs.

Hardware: 5 GPU Architectures

	V100	A40	A100	L40S	H100
Arch	Volta	Ampere	Ampere	Ada	Hopper
SMs	80	84	42	142	132
BW (GB/s)	900	696	968	864	3350
SMEM/SM	96 K	100 K	164 K	100 K	228 K
Ridge	17.4	**53.7**	**7.9**	52.9	20.0
L2 nom.	6	6	20	96	50 MB
L2 eff.	8	**8**	**24**	**112**	**28** MB

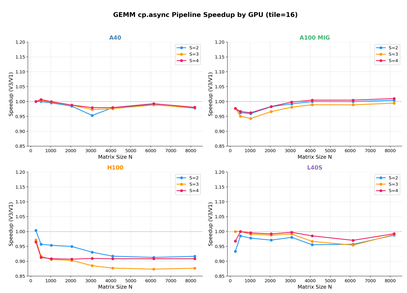
Key: L2 eff. ≠ nominal on *all* GPUs! H100: 50→**28 MB** (slice locality). L40S: 96→**112 MB**.

Model Accuracy

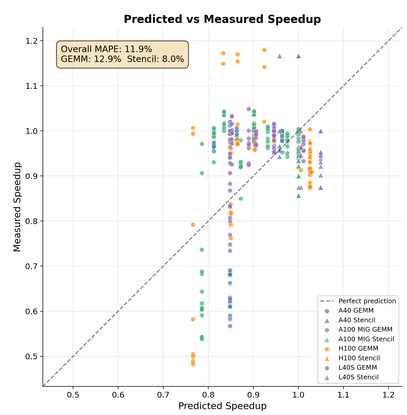
GPU	GEMM	Stencil
A40	11.1%	4.1%
A100 MIG	9.2%	6.7%
L40S	5.7%	7.2%
H100	9.9%	5.5%
Overall	**9.0%**	**5.9%**

Leave-one-out: held-out GPU <15% MAPE. **Zero fitted parameters.**

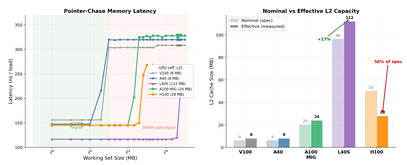
4. Key Results



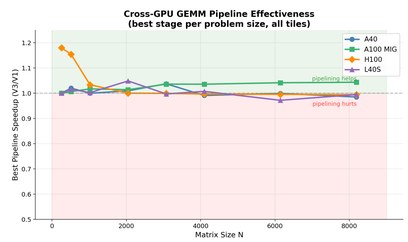
(a) GEMM `cp.async` speedup per GPU



(b) Predicted vs measured (MAPE 11.9%)



(c) L2 characterization: latency & capacity



(d) Cross-GPU best pipeline speedup

5. Key Discoveries & Conclusions

1. **Effective L2 ≠ Nominal L2.** H100: only **28 MB** (not 50 MB) due to L2 slice locality. First documentation of MIG L2 behavior.
2. **280-cycle per-stage overhead floor.** Even 28 FLOP-cycle stencils incur ≥280 cycles of sync overhead. Reduced stencil MAPE: **146%** → **3.7%**.
3. **Sub-linear occupancy loss** (α=0.15): 3→1 blocks/SM costs ∼16%, not 42%.
4. **Parameter-free model achieves ∼9% MAPE** across 4 GPU architectures, 2 kernels, zero fitted coefficients.
5. **Triton pre-filter** prunes `cp.async` stage candidates in autotuning search space.

6. Triton Pre-Filter

```
@autotune.with.cp.async.prefilter(
  configs=[...], key=['M','N','K'],
  workload='gemm', epsilon=0.03)
@triton.jit
def kernel(...):
```

Keeps only stages with $\hat{S} > 1+\epsilon$. Reduces search space, no false negatives.

7. References

[1] NVIDIA, CUDA Prog. Guide §B.24, 2024. [2] CUTLASS, NVIDIA, 2023.
[3] FlashAttention-3, arXiv 2407.08691. [4] Triton, MAPL 2019.
[5] Jia et al., Dissecting GPU Memory, TPDS 2018.

Tools: CMake, SLURM, CUDA 12.x, Nsight Compute, Triton

Platform: UM Great Lakes HPC (V100, A40, A100-MIG, L40S, H100)